



Observabilidade em Microserviços

Material de suporte para entender os três pilares da observabilidade — logs, métricas e rastreamento — e como implementá-los no **vendas-ms** e no **entregas-ms** usando Spring Boot Actuator, Spring Boot Admin e ferramentas do ecossistema.

- ❏ Este material é conceitual e de configuração. O foco é que você consiga **enxergar o sistema por dentro** e saiba onde procurar quando algo der errado em produção.



O Incêndio Silencioso

Imagine acordar às 3h com uma ligação: *"O sistema de vendas está fora do ar. Clientes não conseguem fazer pedidos."* Você abre o terminal. A primeira pergunta que vem à cabeça:

"O que está acontecendo lá dentro?"

Sem logs estruturados, métricas coletadas e rastreamento de requisições, você está a tentar apagar um incêndio no escuro — sem saber onde o fogo começou, o que está a queimar, ou se já se espalhou.

📄 **Observabilidade** é a capacidade de entender o estado interno de um sistema a partir das suas saídas externas. Não é só saber que algo falhou — é saber *exactamente onde, porquê e qual o impacto*.

Por que Microserviços Tornam Tudo Mais Complexo

Em um monólito, há **um processo**. Se cai, você sabe. Em microserviços, uma única acção do utilizador pode tocar **5 componentes diferentes**: vendas-ms → MySQL → ActiveMQ → entregas-ms → MySQL. Se a entrega não foi registada, o erro pode ter ocorrido em qualquer um deles.

Monólito

- Um componente falha → aplicação cai, você sabe
- Rastrear bug → lê o log de um ficheiro
- Deploy com bug → um rollback

Microserviços

- Um serviço degrada **silenciosamente**
- Rastrear bug → correlacionar logs de N serviços
- Deploy com bug → qual serviço foi a origem?

📄 ⚠ Em microserviços, **falhas silenciosas são a norma**. Um serviço pode responder 200 OK enquanto internamente descarta dados. Sem observabilidade, você descobre quando o cliente reclama.

Um avião comercial moderno tem **milhares de sensores**. O piloto não abre a asa para verificar o motor — lê os instrumentos. O seu sistema deve funcionar da mesma forma: cada componente emite **sinais** que permitem entender o estado sem "abrir o capô" em produção.



Os Três Pilares da Observabilidade

A observabilidade é sustentada por três pilares complementares. Nenhum substitui o outro — eles completam-se.



Logs — O QUE aconteceu

Eventos discretos com detalhe rico. Podem ser não estruturados ou estruturados (JSON). A forma mais antiga e intuitiva de observabilidade.



Métricas — QUANTO está acontecendo

Séries temporais com baixo custo de armazenamento. Ideais para alertas e dashboards de saúde do sistema.



Rastreamento — POR ONDE passou

Caminho completo de uma requisição entre serviços. Identifica gargalos e correlaciona spans entre microserviços.

Logs — O Pilar Mais Antigo

Níveis de Log (SLF4J / Logback)

01

TRACE

Detalhes internos — desligado em produção

02

DEBUG

Depuração durante desenvolvimento

03

INFO

"Pedido #123 criado para o cliente João"

04

WARN

Situação inesperada que não causou falha

05

ERROR

Falha que impediu uma operação de concluir

Boas Práticas

Contexto	Inclua IDs, utilizador, valores relevantes
Nível	Use o nível certo para a gravidade
Dados sensíveis	Nunca logue senhas, tokens, CPF
Exceções	<code>log.error("msg", e)</code> — inclua a stack
Volume	INFO e acima em produção
Formato	<code>chave=valor</code> — facilita busca

Log Estruturado em JSON

Activar com `logging.structured.format.console=json`. Permite buscar todos os logs do `pedidoId=4821` em todos os serviços com uma única query — investigação em segundos, não horas.

⚠ Em produção, configure o nível mínimo como **INFO**. DEBUG ligado em produção pode expor dados sensíveis e gerar volume que torna impossível encontrar o que importa.

Métricas, SLI, SLO e SLA

Enquanto logs descrevem *o que aconteceu*, métricas descrevem *quanto, com que frequência e quão rápido*.

Counter

Valores que só crescem. Ex: total de pedidos criados desde o início.

Gauge

Valor actual (sobe e desce). Ex: pedidos em processamento agora.

Histogram

Distribuição de valores. Ex: latência das requisições por percentil.

Timer

Duração de operações. Ex: tempo médio para salvar um pedido.

1

SLI — Medição Real

Taxa de sucesso medida: 99,7%

2

SLO — Meta Interna

Compromisso interno: $\geq 99,5\%$ de sucesso

3

SLA — Contrato Formal

Acordo com penalidades: $\geq 99,0\%$ — abaixo há multa



Error Budget: SLO de 99,5% em 30 dias = **216 minutos de indisponibilidade permitida**. Quando o budget acaba, nenhum novo deploy deve ser feito — a prioridade vira confiabilidade, não features.

Rastreamento Distribuído (Tracing)

Logs dizem o que aconteceu em cada serviço. Métricas dizem o quanto. Mas nenhum responde: **"A requisição do utilizador João que falhou às 14h32 — por qual serviço ela passou e onde travou?"**

Trace

Caminho completo de uma requisição de ponta a ponta entre todos os serviços.

Span

Unidade de trabalho dentro de um trace — uma chamada de método, uma query SQL.

Trace ID

Identificador único propagado entre todos os serviços para correlacionar os spans.

Parent Span

O span que originou o span actual — define a hierarquia da árvore de chamadas.

O Trace ID viaja nos **headers HTTP** entre serviços (padrão W3C Trace Context). Com **Spring Boot 3 + Micrometer Tracing**, essa propagação é **automática** — não é necessário passar o header manualmente. Activar com `management.tracing.sampling.probability=1.0`.

Spring Boot Actuator

O **Spring Boot Actuator** é o ponto de entrada da observabilidade no ecossistema Spring. Expõe **endpoints HTTP** com informações sobre o estado interno da aplicação — como instalar um painel de instrumentos no avião.

Principais Endpoints

<code>/actuator/health</code>	Status UP/DOWN, banco, broker, disco
<code>/actuator/metrics</code>	Lista e valores de métricas
<code>/actuator/prometheus</code>	Métricas no formato Prometheus
<code>/actuator/loggers</code>	Nível de log de cada pacote
<code>/actuator/env</code>	Variáveis de ambiente e propriedades
<code>/actuator/threaddump</code>	Dump das threads em execução
<code>/actuator/heapdump</code>	Dump do heap JVM para análise offline

Recursos Essenciais

→ Health Check

Agrega o estado de todos os componentes críticos (banco, broker, disco). Se o banco cair, o status muda para DOWN e orquestradores param de enviar tráfego automaticamente.

→ Log Level em Tempo Real

Mude o nível de log de qualquer pacote **sem reiniciar a aplicação** via POST no `/actuator/loggers`. Ouro em incidentes de produção.

→ Métricas Customizadas com Micrometer

Crie `Counter`, `Gauge` e `Timer` de negócio directamente no código — ex: `vendas.pedidos.criados`, `vendas.pedidos.falhos`.

- ❏ Configure uma **porta separada** para o Actuator (`management.server.port=8090`) para não misturar com o tráfego da aplicação.

Spring Boot Admin

O **Spring Boot Admin** é uma aplicação Spring Boot separada que descobre e agrega as informações do Actuador de múltiplos serviços, apresentando tudo em uma interface web amigável. É a diferença entre ler JSON bruto e ter um painel visual.



Dashboard & Health

Lista todos os serviços com status UP/DOWN/OFFLINE. Detalhes de saúde de cada componente com visual colorido.



Métricas em Tempo Real

Gráficos de memória, CPU, threads e latência. Altere o nível de log via interface gráfica — sem `curl`.



Notificações

Alertas por email/Slack quando um serviço cai ou sobe. Transforma o seu laptop em uma central de operações.

- ❏ Configure o cliente no vendas-ms com `spring.boot.admin.client.url=http://localhost:9090` e `management.endpoints.web.exposure.include=*` para o Admin ver tudo.

Ferramentas do Ecossistema

Open Source

Prometheus	Colecta métricas em série temporal via <code>/actuator/prometheus</code>
Grafana	Dashboards unificados sobre Prometheus, Loki e Zipkin
Loki	Agregação e busca de logs, indexa por labels
Zipkin	Colecta e visualiza traces distribuídos por Trace ID
Jaeger	Alternativa ao Zipkin, mais robusta, suporte OpenTelemetry
OpenTelemetry	Padrão vendor-neutral para logs, métricas e traces
ELK Stack	Elasticsearch + Logstash + Kibana para busca full-text

Comercial

Datadog	APM full-stack, fácil integração, por host/mês
New Relic	Method-level tracing, baseado em dados
Dynatrace	Descoberta automática de topologia, IA para anomalias
Splunk	Busca de logs em grande escala, padrão enterprise
Honeycomb	Alta cardinalidade, exploração muito rápida
AWS CloudWatch	Nativo AWS, sem fricção se já está na AWS



Não existe resposta errada. Muitas empresas usam **Grafana + Prometheus + Loki** open source para métricas e logs, e uma ferramenta comercial de APM para tracing — combinando o melhor dos dois mundos.

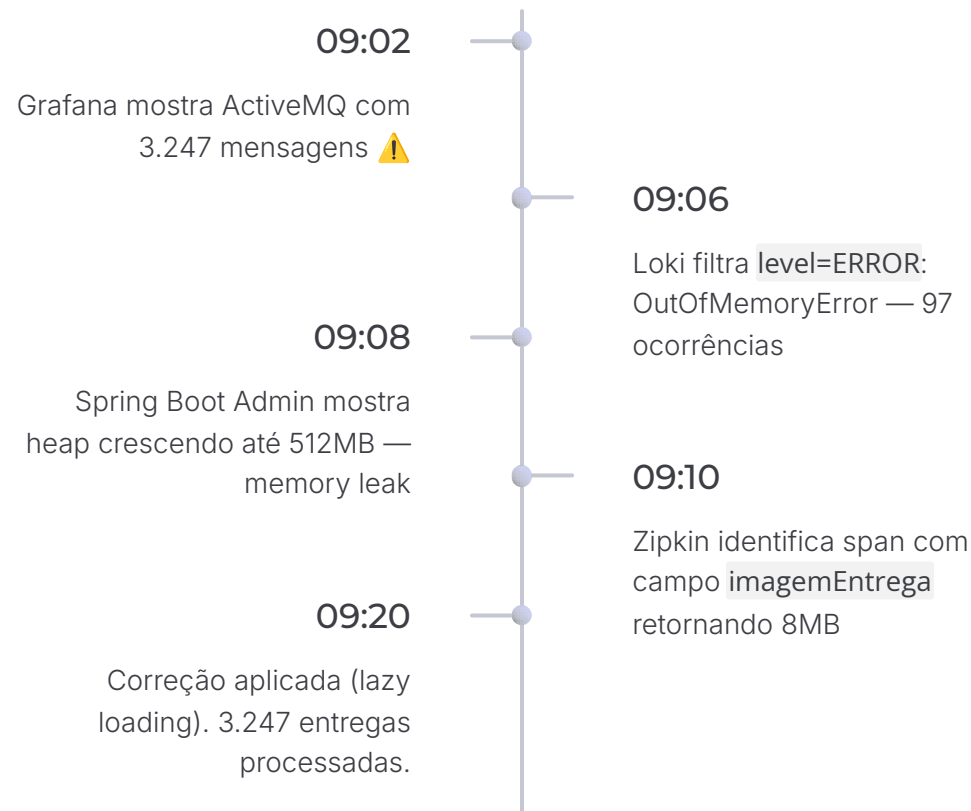
Estudo de Caso: Investigando um Incidente Real

Cenário: Segunda-feira às 9h. Clientes conseguem criar pedidos, mas as entregas não aparecem no entregas-ms.

❌ Sem Observabilidade — 2h30 de downtime



✅ Com Observabilidade — 40 minutos



Docker Compose: Stack de Observabilidade

Levante toda a stack de observabilidade com um único ficheiro `docker-compose-observabilidade.yml`:



Prometheus :9090

Colecta métricas dos serviços a cada 15 segundos via `/actuator/prometheus`



Grafana :3000

Dashboards unificados. Acesso com `admin/admin`. Conecta ao Prometheus, Loki e Zipkin.



Zipkin :9411

Colecta e visualiza traces distribuídos. Interface web para buscar por Trace ID.



Spring Admin :9000

Gestão visual dos microserviços. Imagem do seu admin-server personalizado.

O ficheiro `prometheus.yml` configura o scrape dos serviços: `host.docker.internal:8090` para o vendas-ms e `host.docker.internal:8091` para o entregas-ms, ambos no path `/actuator/prometheus`.

Mapa de Implementação

vendas-ms

<code>pom.xml</code>	Adicionar actuator, micrometer-prometheus, admin-client
<code>application.properties</code>	Endpoints Actuator, porta de management, registo no Admin
<code>PedidoService.java</code>	Logs estruturados e métricas customizadas com Micrometer

entregas-ms

<code>pom.xml</code>	Mesmas dependências do vendas-ms
<code>application.properties</code>	Porta de management 8091, registo no Admin
<code>EntregaService.java</code>	Logs e métricas no consumer de mensagens

Admin Server (novo projecto)

<code>pom.xml</code>	spring-boot-admin-starter-server
<code>AdminServerApplication.java</code>	Classe principal com <code>@EnableAdminServer</code>
<code>application.properties</code>	Porta 9090, nome admin-server

Infra (Docker Compose)

<code>docker-compose-observabilidade.yml</code>	Prometheus, Grafana, Zipkin, Spring Boot Admin
<code>prometheus.yml</code>	Scrape do vendas-ms e entregas-ms

Monitoramento vs Observabilidade

Muito material usa os dois termos como sinónimos. Não são.

Monitoramento

Colecta e visualiza métricas **predefinidas**. Você decide antes o que quer observar e configura alertas para isso.

- Foco: detectar falhas conhecidas
- Pergunta: "Está quebrado?"
- Abordagem: reactiva (alerta dispara)
- Analogia: termómetro de febre

Observabilidade

Permite fazer **perguntas arbitrárias** sobre o estado interno, incluindo perguntas que você não sabia que precisaria fazer.

- Foco: entender falhas desconhecidas
- Pergunta: "Por que quebrou? Onde? Para quem?"
- Abordagem: exploratória (investigação activa)
- Analogia: exame de sangue completo + raio-X

  Monitoramento é **necessário mas não suficiente**. Todo sistema observável pode ser monitorado, mas nem todo sistema monitorado é observável. Monitoramento é um subconjunto de observabilidade.

Checklist de Aprendizado

Fundamentos

- Sei explicar a diferença entre **monitoramento** e **observabilidade** com um exemplo concreto
- Entendo os três pilares: **logs**, **métricas** e **rastreamento**, e quando cada um é mais útil
- Sei escrever um **log estruturado** com contexto adequado sem expor dados sensíveis

Spring Boot Actuator & Admin

- Sei adicionar o **Actuator** e configurar os endpoints expostos
- Sei mudar o nível de log em tempo real **sem reiniciar** a aplicação
- Consigo criar **métricas customizadas** com Counter e Timer usando Micrometer
- Sei configurar o **Spring Boot Admin** (servidor e cliente)

SLI, SLO, SLA e Error Budget

- Sei distinguir **SLI**, **SLO** e **SLA** e construir um exemplo para o vendas-ms
- Entendo o conceito de **error budget** e por que ele governa decisões de deploy

Ecossistema & Tracing

- Entendo o que é um **Trace ID** e como correlaciona spans entre serviços
- Sei diferenciar: **Prometheus** (colecta), **Grafana** (visualização), **Zipkin** (tracing), **Loki** (logs)
- Consigo articular por que **observabilidade é crítica em microserviços** com o estudo de caso